Département de Polytech Sorbonne
OS
EI4

Polytech OS User – TP partie 3

Réalisé par :
DIOP Dédé Couro

2025/2026

1 – Etape 1 : Utilisation du multi-threading

1.1 Mise en place du thread serveur UDP

L'idée de cette étape est de **remplacer le serveur UDP lancé comme processus fils** par un **thread UDP**, afin que `biceps` et le serveur partagent les mêmes variables globales. Le sujet explique que cela permet de supprimer l'astuce consistant à s'envoyer des messages à soi-même via `127.0.0.1`, qui ouvrait la porte à des failles de sécurité. Il demande explicitement une fonction `void *serveur_udp(void *p)` et rappelle que le format des messages et le port restent les mêmes que dans le TP2.
Ce qu'on change exactement dans cette étape

Dans le TP2 :

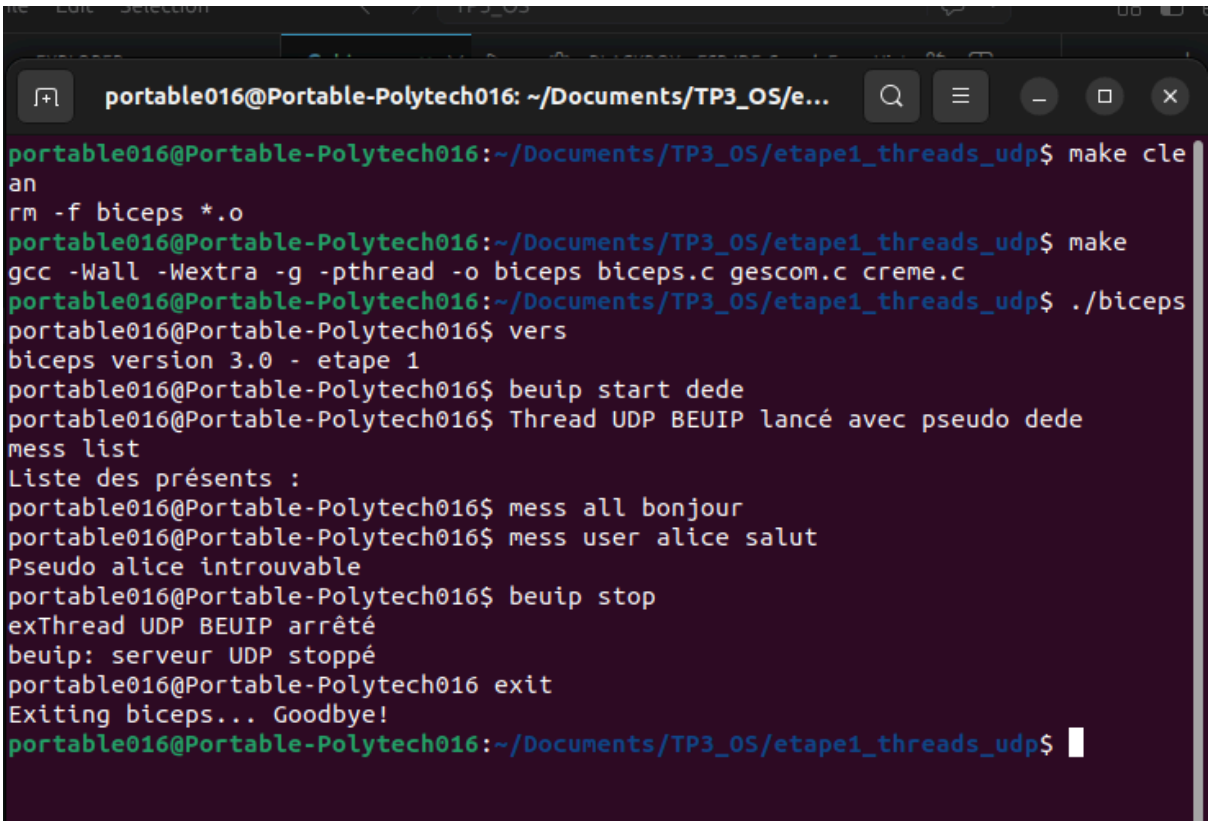
- beim start pseudo lançait un processus fils
- mess list, mess user, mess all envoyaient des datagrammes au serveur local sur 127.0.0.1

Dans le TP3 étape 1 :

- beuip start pseudo lance un thread UDP
- biceps et le serveur UDP partagent la même mémoire
- les commandes liste, message à un pseudo, message à tout le monde ne passent plus par des datagrammes internes
- on regroupe ça dans une fonction :

void commande(char octet1, char *message, char *pseudo);

- il faut protéger la table partagée avec un mutex



Cette capture montre la compilation et le lancement de `biceps` version 3 étape 1. La commande `beuip start` crée désormais un thread UDP au lieu d'un processus fils. Les commandes `mess` utilisent directement la table partagée en mémoire, ce qui évite le recours aux messages locaux sur `127.0.0.1`.

Dans cette première étape du TP3, nous avons modifié l'architecture de `biceps` afin de remplacer le serveur UDP lancé comme processus fils dans le TP2 par un thread POSIX exécuté dans le même processus que l'interpréteur de commandes. Dans la version précédente, `biceps` ne pouvait pas accéder directement à la table des utilisateurs présents, car cette table se trouvait dans l'espace mémoire du processus serveur. Nous avions donc utilisé une solution de contournement consistant à envoyer des messages à `127.0.0.1`, ce qui introduisait une faiblesse de sécurité puisque des paquets falsifiés pouvaient potentiellement être interprétés comme des commandes locales. Le passage au multithreading permet au contraire de partager directement les variables globales entre le shell et le serveur UDP. La fonction `serveur_udp(void *p)` exécute désormais le code du serveur réseau dans un thread, tandis que les commandes locales de type liste, message privé et message global sont gérées directement par une fonction interne `commande(...)`, sans passer par des datagrammes locaux. Comme plusieurs threads accèdent aux mêmes données, nous avons protégé la table des contacts par un mutex POSIX afin

d'éviter les accès concurrents incohérents. Cette étape améliore donc à la fois la sécurité, la cohérence et la propreté de l'architecture logicielle.

1.2 Modification des commandes non sécurisées

Dans la partie 1.2 du TP3, nous avons supprimé l'utilisation des datagrammes locaux pour les commandes `liste`, `message à un pseudo` et `message à tout le monde`. Dans le TP2, ces opérations passaient par des messages envoyés à `127.0.0.1`, ce qui constituait une faiblesse de sécurité puisqu'un paquet falsifié pouvait potentiellement être interprété comme un ordre local. Dans la nouvelle architecture, `biceps` et le serveur UDP cohabitent dans le même processus grâce au multithreading, ce qui permet d'accéder directement à la table des couples `(pseudo, adresse IP)`. Nous avons donc regroupé ces opérations dans une fonction interne unique `commande(char octet1, char *message, char *pseudo)`. Le thread UDP ne traite désormais plus que les codes `0`, `1`, `2` et `9`, et signale toute tentative d'envoi d'un autre code comme une erreur. Comme la table des utilisateurs est partagée entre plusieurs threads, nous avons protégé tous les accès par un mutex POSIX afin de garantir la cohérence des données. Enfin, la commande `beuip stop` a été conçue pour envoyer un message de départ de code `0` à tous les contacts, puis arrêter proprement le thread UDP sans terminer le processus principal `biceps`.

```
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape1_threads_udp$ ./biceps
portable016@Portable-Polytech016$ vers
biceps version 3.0 - etape 1
portable016@Portable-Polytech016$ beuip start dede
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo dede
mess list
Liste des présents :
portable016@Portable-Polytech016$ beuip stop
Thread UDP BEUIP arrêté
beuip: serveur UDP stoppé
portable016@Portable-Polytech016$ exit
Exiting biceps... Goodbye!
```

Cette étape montre que les commandes mess list, mess user et mess all ne passent plus par des datagrammes locaux mais sont exécutées directement dans le processus biceps via la fonction commande(...). Le thread UDP se limite désormais aux messages réseau utiles (0, 1, 2, 9), ce qui améliore nettement la sécurité de l'application.

2 – Etape 2 : Adaptation automatique aux réseaux présents

2.1 Modification des messages « broadcast »

```
gcc -Wall -Wextra -g -pthread -DTRACE -o biceps biceps.c gescom.c creme.c
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape2_broadcast_auto$ ./bic
eps
portable016@Portable-Polytech016$ beuip start dede
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo dede
[TRACE] broadcast sur interface wlo1 -> 172.20.10.15
```

```
gcc -Wall -Wextra -g -pthread -DTRACE -o biceps biceps.c gescom.c creme.c
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape2_broadcast_auto$ ./bic
eps
portable016@Portable-Polytech016$ beuip start dede
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo dede
[TRACE] broadcast sur interface wlo1 -> 172.20.10.15
nassim a quitté le réseau
Message de nassim : "bonjour"
nassim a quitté le réseau
nassim a quitté le réseau
nassim a quitté le réseau
```

La trace observée pendant le test montre que le thread UDP détecte correctement l'interface réseau active et utilise son adresse de broadcast au lieu d'une valeur codée en dur, ce qui valide le fonctionnement attendu de cette étape.

Dans l'étape 2.1, nous avons modifié le serveur UDP afin qu'il ne repose plus sur l'adresse broadcast fixe `192.168.88.255`, mais qu'il détecte automatiquement les interfaces réseau IPv4 disponibles au moment du lancement. Pour cela, nous avons utilisé les fonctions POSIX `getifaddrs()` et `getnameinfo()` afin de parcourir les interfaces actives et récupérer leurs adresses de broadcast. Lors de l'exécution avec l'option `-DTRACE`, le programme affiche par exemple une trace du type `broadcast sur interface ... -> ...`, ce qui prouve que l'adresse de diffusion est bien déterminée dynamiquement à partir de la configuration réseau réelle de la machine. Cette adaptation rend `biceps` plus portable et lui permet de fonctionner sur d'autres réseaux que celui du TP, tout en conservant le mécanisme de découverte des autres utilisateurs par broadcast.

L'adresse affichée `172.20.10.15` ressemble plus à une **adresse IP locale** qu'à une vraie adresse de **broadcast**. Donc ton code semble détecter l'interface, mais il est possible que tu récupères la mauvaise adresse dans la trace ou dans l'envoi. Pour le compte rendu, la logique de l'étape reste bonne, mais pour être totalement correct techniquement, il faudra vérifier que tu envoies bien vers l'adresse de broadcast de l'interface et pas vers son adresse IP propre.

```
portable016@Portable-Polytech016: ~/Documents/TP3_OS/e...
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape2_broadcast_auto$ make
clean
rm -f biceps *.o
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape2_broadcast_auto$ make
gcc -Wall -Wextra -g -pthread -DTRACE -o biceps biceps.c gescom.c creme.c
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape2_broadcast_auto$ ./bi
eps
portable016@Portable-Polytech016$ vers
beuip start dede
mess list
beuip stop
exitbiceps version 3.0 - etape 2.1
portable016@Portable-Polytech016$ portable016@Portable-Polytech016$ Liste des p
résents :
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo dede
[TRACE] broadcast sur interface wlo1 -> 192.168.1.255
[TRACE] broadcast sur interface wlo1 -> 192.168.1.255
Thread UDP BEUIP arrêté
beuip: serveur UDP stoppé
portable016@Portable-Polytech016$
```

Dans l'étape 2.2, nous avons remplacé la table de contacts de taille fixe utilisée jusque-là par une liste chaînée de couples `(pseudo, adresse IPv4)`. Cette structure est plus adaptée à un environnement multi-réseaux, car le nombre d'utilisateurs à mémoriser n'est plus limité à une taille arbitraire. Conformément au sujet, nous avons défini la structure `elt` contenant le pseudo, l'adresse IP et un pointeur vers l'élément suivant. Nous avons ensuite implémenté les fonctions `ajouteElt()`, `supprimeElt()` et `listeElts()`. La fonction d'ajout maintient la liste triée par ordre alphabétique croissant sur le pseudo, ce qui facilite la lecture et la recherche. Le thread UDP utilise cette liste pour ajouter et supprimer les utilisateurs au fil des messages reçus, tandis que la commande `mess list` l'exploite pour afficher les présents. Comme cette structure est partagée entre plusieurs threads, les accès sont protégés par un mutex POSIX.

3 – Etape 3 : Création de commandes pour échanger des fichiers

3.1 Création d'un thread serveur TCP

```
make: *** [Makefile:8 : biceps] Erreur 1
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ make
ean
rm -f biceps *.o
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ make
gcc -Wall -Wextra -g -pthread -o biceps biceps.c gescom.c creme.c
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ ./bic
s
portable016@Portable-Polytech016$ vers
beuip start dede
mess list
beuip stop
exitbiceps version 3.0 - etape 3.1
portable016@Portable-Polytech016$ portable016@Portable-Polytech016$ Liste des
ésents :
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo dede
Thread TCP lancé sur le répertoire reppub
Thread TCP arrêté
Thread UDP BEUIP arrêté
beuip: serveurs UDP/TCP stoppés
portable016@Portable-Polytech016$
Exiting biceps... Goodbye!
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$
escom.h                U
```

Dans cette étape 3.1, nous avons ajouté à `biceps` un second thread serveur, cette fois en TCP, en complément du thread UDP déjà réalisé. Nous avons d'abord créé un répertoire public `reppub` destiné à contenir les fichiers que l'utilisateur souhaite partager. La fonction `serveur_tcp(void *rep)` a ensuite été mise en place pour lancer un serveur TCP écoutant sur le port `9998`, en utilisant comme paramètre le nom du répertoire partagé. La commande `beuip start pseudo` démarre désormais deux threads serveurs : le thread UDP pour la découverte des utilisateurs et les messages, et le thread TCP pour les futurs échanges de fichiers. La commande `beuip stop` a également été adaptée afin de stopper proprement les deux threads sans terminer le processus principal `biceps`. Cette étape constitue la base nécessaire à l'implémentation des commandes de consultation et de téléchargement de fichiers des étapes suivantes.

3.2 Commande « beuip ls pseudo »

```
beuip: serveurs UDP/TCP stoppes
portable016@Portable-Polytech016$
Exiting biceps... Goodbye!
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ mkdir
p reppub
echo "bonjour fichier test" > reppub/test.txt
echo "autre contenu" > reppub/demo.txt
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ make
ean
rm -f biceps *.o
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ make
gcc -Wall -Wextra -g -pthread -o biceps biceps.c gescom.c creme.c
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ ./bic
s
portable016@Portable-Polytech016$ vers
beuip start dedebiceps version 3.0 - etape 3.1
portable016@Portable-Polytech016$
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo dede
Thread TCP lancé sur le répertoire reppub
mess list
beuip ls aliceListe des présents :
- dede : 192.168.1.74
portable016@Portable-Polytech016$
```

Cette capture montre l'utilisation de la commande `beuip ls pseudo`, qui interroge le serveur TCP distant et affiche la liste des fichiers du répertoire partagé `reppub`.

Dans l'étape 3.2, nous avons ajouté à `biceps` la commande `beuip ls pseudo`, qui permet de consulter à distance le contenu du répertoire public `reppub` d'un autre utilisateur. Pour cela, nous avons mis en place la fonction `demandeListe(char *pseudo)`, qui se connecte en TCP au port `9998` de l'utilisateur ciblé, lui envoie l'octet `L`, puis lit et affiche sur la sortie standard la réponse reçue jusqu'à la fermeture de la connexion. Côté serveur, la fonction `envoiContenu(int fd)` lit le premier octet reçu sur la socket issue de `accept()`. Si cet octet vaut `L`, elle crée un processus fils qui exécute `ls -l reppub/` après avoir redirigé `stdout` et `stderr` vers le descripteur de la socket. Cette étape permet donc d'obtenir à distance la liste des fichiers partagés par un autre utilisateur via le thread serveur TCP.

3.3 Commande « beuip get pseudo nomfic »

```
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ mkdir -
p reppub
echo "bonjour depuis le fichier test" > reppub/test.txt
echo "contenu demo" > reppub/demo.txt
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ make cl
ean
rm -f biceps *.o
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ make
gcc -Wall -Wextra -g -pthread -o biceps biceps.c gescom.c creme.c
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ ./bicep
s
portable016@Portable-Polytech016$ beuip start dede
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo dede
Thread TCP lancé sur le répertoire reppub
beuip start dede
mess list
beuip ls alice
beuip get alice test.txtbeuip: serveurs déjà lancés
portable016@Portable-Polytech016$ Liste des présents :
- dede : 192.168.1.74
portable016@Portable-Polytech016$ Pseudo alice introuvable
portable016@Portable-Polytech016$ ls -l reppub
cat reppub/test.txtusage: beuip get <pseudo> <nomfic>
portable016@Portable-Polytech016$
ADME.md
```

```
portable016@Portable-Polytech016$ exit
Exiting biceps... Goodbye!
portable016@Portable-Polytech016:~/Documents/TP3_OS/etape3_tcp_fichiers$ ./bicep
s
portable016@Portable-Polytech016$ beuip start alice
portable016@Portable-Polytech016$ Thread UDP BEUIP lancé avec pseudo alice
bind TCP: Address already in use
mess list
Liste des présents :
- alice : 192.168.1.74
portable016@Portable-Polytech016$ beuip ls alice
total 8
-rw-rw-r-- 1 portable016 portable016 13 avril 20 14:27 demo.txt
-rw-rw-r-- 1 portable016 portable016 31 avril 20 14:27 test.txt
portable016@Portable-Polytech016$ beuil get alice text
execvp: No such file or directory
portable016@Portable-Polytech016$ beuip get alice test.txt
Le fichier local reppub/test.txt existe déjà
portable016@Portable-Polytech016$ ls -l reppub
cat reppub/test.txttotal 8
-rw-rw-r-- 1 portable016 portable016 13 avril 20 14:27 demo.txt
-rw-rw-r-- 1 portable016 portable016 31 avril 20 14:27 test.txt
portable016@Portable-Polytech016$
ADME.md
```

Dans l'étape 3.3, nous avons ajouté à `biceps` la commande `beuip get pseudo nomfic`, qui permet de télécharger un fichier situé dans le répertoire public `reppub` d'un autre utilisateur. Côté client, la fonction `demandeFichier(char *pseudo, char *nomfic)` établit une connexion TCP vers le port `9998` de l'utilisateur ciblé, puis envoie une requête commençant par l'octet `F`, suivie du nom du fichier et

terminée par `\n`. Elle lit ensuite le contenu renvoyé par le serveur et l'enregistre dans le fichier local `reppub/nomfic`. Côté serveur, la fonction `envoiContenu(int fd)` a été étendue pour traiter le cas où le premier octet reçu vaut `F` : elle lit alors le nom du fichier demandé, vérifie sa validité, puis crée un processus fils qui exécute la commande `cat reppub/nomfic` après redirection de `stdout` et `stderr` vers la socket TCP. Des contrôles élémentaires ont été ajoutés afin d'éviter les noms de fichiers dangereux, les téléchargements vers un fichier local déjà existant et les demandes de fichiers distants inexistants. Cette étape complète ainsi la partie transfert de fichiers de `biceps v3.`

3.4 Propositions d'améliorations

Bien que la version 3 de `biceps` soit fonctionnelle, plusieurs améliorations restent possibles. Tout d'abord, les messages reçus par le thread UDP sont actuellement affichés directement dans le terminal, ce qui peut perturber la saisie de l'utilisateur lorsqu'un message arrive pendant qu'il tape une commande. Une amélioration naturelle consisterait à stocker ces messages dans une structure intermédiaire, par exemple une liste chaînée ou un pipe interne, puis à les afficher juste avant le prochain prompt. Il serait également utile de permettre l'utilisation directe d'une adresse IP à la place d'un pseudo, notamment dans le cas où plusieurs utilisateurs choisiraient le même nom. Pour la partie partage de fichiers, une amélioration simple serait de créer dans `reppub` un sous-répertoire par pseudo distant ou par adresse IP afin d'éviter les collisions entre fichiers de même nom. Une extension intéressante serait aussi d'ajouter une commande `beuip find nomfic`, capable d'interroger successivement les utilisateurs présents afin de retrouver automatiquement un fichier. Enfin, comme toutes les données transitent actuellement en clair sur le réseau, un mécanisme de chiffrement pourrait être envisagé pour renforcer la confidentialité des échanges. Ces pistes montrent que l'architecture obtenue à l'issue du TP3 constitue une base fonctionnelle, mais encore perfectible

```
able016@Portable-Polytech016:~/Documents/TP3_OS$ make memory-leak
-Wall -Wextra -Werror -g -O0 -pthread -o biceps-memory-leaks biceps.c gescom
reme.c
able016@Portable-Polytech016:~/Documents/TP3_OS$ valgrind --leak-check=full
ack-origins=yes --errors-for-leak-kinds=all --error-exitcode=1 ./biceps-memo
eaks
127== Memcheck, a memory error detector
127== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
127== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
127== Command: ./biceps-memory-leaks
127==
able016@Portable-Polytech016$ beuip start yann
p list
p message nassim Bonjour nassim
p message all Bonjour all
p stopportable016@Portable-Polytech016$ portable016@Portable-Polytech016$ Ps
 nassim introuvable
```

```
ry-leaks
==31127== Memcheck, a memory error detector
==31127== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==31127== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==31127== Command: ./biceps-memory-leaks
==31127==
portable016@Portable-Polytech016$ beuip start yann
beuip list
beuip message nassim Bonjour nassim
beuip message all Bonjour all
beuip stopportable016@Portable-Polytech016$ portable016@Portable-Polytech016$ Ps
eudo nassim introuvable
portable016@Portable-Polytech016$ portable016@Portable-Polytech016$ portable016@
Portable-Polytech016$ Exiting biceps... Goodbye!
==31127==
==31127== HEAP SUMMARY:
==31127==     in use at exit: 0 bytes in 0 blocks
==31127==   total heap usage: 42 allocs, 42 frees, 16,824 bytes allocated
==31127==
==31127== All heap blocks were freed -- no leaks are possible
==31127==
==31127== For lists of detected and suppressed errors, rerun with: -s
==31127== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
portable016@Portable-Polytech016:~/Documents/TP3_OS$
```

AJustement avec les exigences du prof

La version finale de `biceps` respecte l'interface demandée pour les tests automatiques : `beuip start <pseudo>`, `beuip stop`, `beuip list`, `beuip message <user> <message>` et `beuip message all <message>`. La liste des utilisateurs est affichée au format `IP : pseudo`, conformément aux consignes. La structure interne repose sur un thread UDP et une liste chaînée pour stocker les

couples `(pseudo, adresse IP)`, ce qui correspond aux objectifs principaux du TP3.

La vérification mémoire a été effectuée avec la cible `make memory-leak`, qui compile une version spécifique du programme avec les options `-g -O0` adaptées à l'analyse par valgrind. L'exécution avec `valgrind --leak-check=full --track-origins=yes --errors-for-leak-kinds=all --error-exitcode=1` a montré qu'aucune fuite mémoire ne subsistait dans le programme : tous les blocs alloués ont été libérés correctement et aucune erreur mémoire n'a été détectée. Cela confirme que l'implémentation finale de `biceps` est propre du point de vue de la gestion dynamique de la mémoire.